

Object-Oriented Programming

Placement Revision Cheatsheet

Concept-first revision notes for SDE fresher interviews. Tag pills next to headings tell you how often each topic is asked:

Tag	What it means
MUST KNOW	Asked in 90%+ of OOP interviews. Don't skip.
HIGH FREQ	Asked in ~50% of OOP interviews.
AMAZON FAV	Topics Amazon-style interviewers love.
SERVICE-BASED FAV	Favored by TCS, Infosys, Wipro, Cognizant, Capgemini.
GFG CLASSIC	Frequently appears as a standalone GeeksforGeeks question.
TRICKY	Easy to get wrong if you only know the surface answer.
RARE BUT IMPRESSIVE	Rarely asked, but stuns interviewers when you know it.

■ **Analogy:** Yellow callout box: maps an OOP concept to an everyday situation.

★ **Interview Gold:** Teal callout box: the high-leverage insight on a topic.

Table of Contents

- 1. The Big Picture** — Why OOP exists, four pillars, terminology
- 2. Classes & Objects** — Anatomy, constructors, destructors, this/self
- 3. Encapsulation** — Access modifiers, getters/setters, data hiding
- 4. Inheritance** — IS-A, types, diamond problem, composition vs inheritance
- 5. Polymorphism** — Overloading vs overriding, vtables, dynamic dispatch
- 6. Abstraction** — Abstract classes vs interfaces — when to use what
- 7. SOLID Principles** — Five rules every interviewer expects you to know
- 8. Common Design Patterns** — Singleton, Factory, Observer, Strategy, Decorator
- 9. OOP Across Languages** — C++ vs Java vs Python — what's different
- 10. Interview Rapid-Fire** — Top 30 questions, tricky distinctions, last-day revision

1. The Big Picture

MUST KNOW

OOP is a way of organizing code around "things" (objects) that have state and behavior. Procedural code organizes around functions that operate on data. OOP bundles the data and the functions that operate on it together. That's the entire shift.

■ **Analogy:** Procedural code is a recipe book — separate instructions on a page that act on ingredients in a bowl. OOP is a smart kitchen appliance — the data (food) and behaviors (chop, mix, heat) live inside one device. You ask the device 'make a smoothie' and it knows what's inside and what to do.

1.1 Why OOP exists

- **Manage complexity** — break big problems into small, self-contained objects.
- **Reuse code** — inheritance and composition let you build new things from old.
- **Hide details** — users of an object don't need to know how it works.
- **Model the real world** — objects in code map to objects in the problem domain.
- **Change safely** — change an object's internals without breaking everyone who uses it.

1.2 The Four Pillars

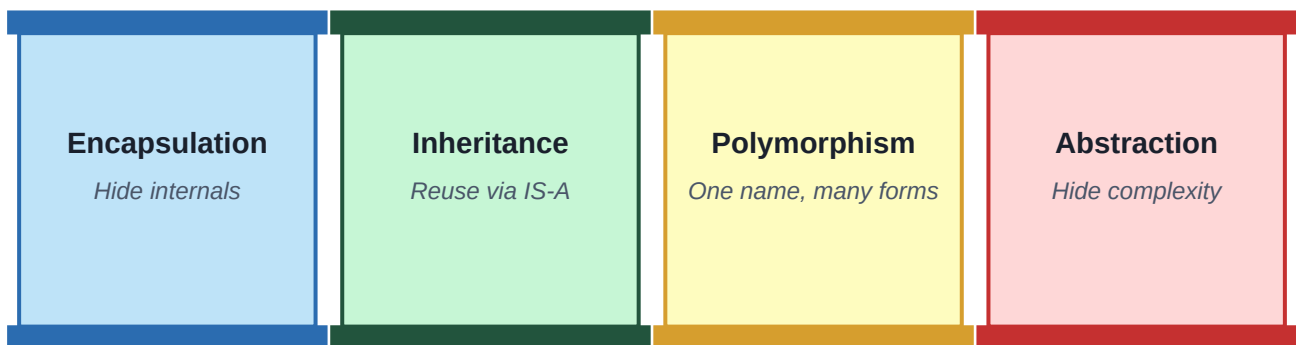
MUST KNOW

HIGH FREQ

SERVICE-BASED FAV

Every OOP interview opens with these. Memorize them as a one-line definition each:

The Four Pillars of OOP



All four together = real object-oriented design

Pillar	One-line definition	Real-world example
Encapsulation	Bundle data and methods; hide internal state behind an interface.	ATM — you see buttons, not the cash mechanism.
Inheritance	A new class derives state and behavior from an existing class (IS-A).	Dog IS-A Animal — inherits eat(), sleep().
Polymorphism	One interface, many implementations — picked at compile or runtime.	Different shapes all implement area().

Abstraction	Expose only essential features; hide complexity.	Steering wheel — you turn it, don't think about hydraulics.
-------------	--	---

★ **Interview Gold: The trick to remembering them:** three pillars describe what OOP *does* (encapsulate, inherit, be polymorphic), and one describes the *goal* (abstraction — hiding complexity). Abstraction is the "why"; the other three are the "how".

1.3 Core terminology

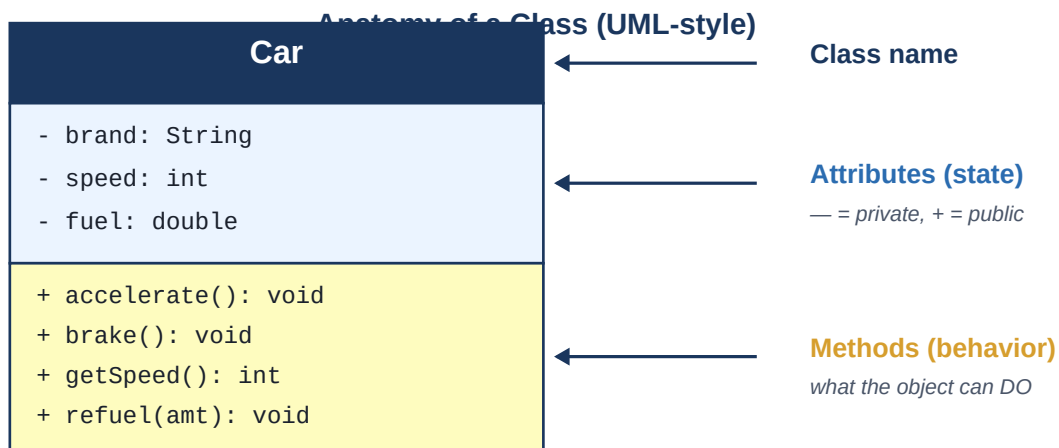
Term	Meaning
Class	A blueprint or template defining state and behavior.
Object	An instance of a class — a real thing in memory.
Field / Attribute	A variable that holds state inside an object.
Method	A function defined inside a class; operates on the object's state.
Constructor	Special method that initializes a new object.
Destructor / Finalizer	Special method called when an object is destroyed.
Instance	Another word for object.
Reference / Pointer	A handle to an object — not the object itself.
Member	Generic term for fields and methods of a class.

★ **Interview Gold: Class vs Object** is the most-asked fresher question of all time. Class = recipe. Object = the actual cake. You can have one recipe and bake 100 cakes. The class lives once in your code; objects are created as many times as you need at runtime.

2. Classes & Objects

MUST KNOW **HIGH FREQ**

A **class** defines what an object will be made of and what it can do. An **object** is a concrete instance — created at runtime, occupying memory.



2.1 Class anatomy

```
class Car {                                // class definition
    private String brand;                  // field (state)
    private int speed;

    public Car(String b) {                 // constructor
        this.brand = b;
        this.speed = 0;
    }

    public void accelerate() {             // method (behavior)
        speed += 10;
    }
}

Car c = new Car("Tesla");                 // create an object
```

2.2 Constructors

HIGH FREQ GFG CLASSIC

A **constructor** runs automatically when an object is created. Its job is to initialize the object's state. Same name as the class, no return type.

- **Default constructor:** no arguments. If you don't write one, the compiler often provides one for free.
- **Parameterized constructor:** takes arguments to initialize fields.
- **Copy constructor (C++):** takes another object and copies its state. Java/Python use clone/copy methods instead.
- **Constructor overloading:** multiple constructors with different parameters. Compile-time polymorphism.

★ **Interview Gold: Why no return type?** The constructor implicitly returns a fully-initialized object. Writing a return type would make it a regular method. `void Car()` is NOT a constructor — it's a method named `Car`.

2.3 Destructors / Finalizers

Language	Mechanism	When it runs
C++	~ClassName() destructor	Object goes out of scope or delete called.
Java	finalize() (deprecated)	Before GC reclaims — unpredictable. Avoid.
Python	__del__()	When ref count → 0, but timing isn't guaranteed.
Modern	RAII / try-with-resources / context managers	Deterministic cleanup, language-supported.

★ **Interview Gold:** Java's finalize() is deprecated since Java 9 — it runs at unpredictable times and isn't guaranteed to run at all. Modern Java uses **try-with-resources** and AutoCloseable for cleanup. Same idea in Python: with + __enter__ / __exit__.

2.4 The this / self reference

HIGH FREQ TRICKY

Inside an instance method, this (Java/C++) or self (Python) refers to the object the method was called on. It disambiguates between a field and a parameter with the same name.

```
class Person {
    String name;
    public Person(String name) {
        this.name = name;    // 'this.name' = field, 'name' = parameter
    }
}
```

■ **Analogy:** this is like the word "I" — every person uses the same word, but it points to a different individual depending on who's speaking. Every object running a method says "I" to mean itself.

2.5 Static (class-level) members

HIGH FREQ AMAZON FAV

Normal fields and methods belong to *each object*. **Static** members belong to the *class* itself — shared by all instances, accessed without creating an object.

```
class Counter {
    static int totalCount = 0;    // shared across all objects
    int instanceId;              // per-object

    Counter() {
        totalCount++;
        instanceId = totalCount;
    }
}
Counter.totalCount    // accessed via class, not object
```

★ **Interview Gold:** Two interview traps with static:

- (1) Static methods **cannot use this** — there's no specific object they belong to.
 - (2) Static methods **cannot directly access instance fields** for the same reason.
- Static = class-level, instance = object-level. Don't mix them up.

3. Encapsulation

MUST KNOW **HIGH FREQ** **SERVICE-BASED FAV**

Encapsulation = bundling state + behavior together AND hiding the state behind methods. Outside code can only interact with the object through its public methods — not by touching fields directly.

■ **Analogy:** Think of a capsule of medicine. The active ingredient (state) is wrapped inside a shell, and you interact with it only through swallowing (the public method). You can't reach in and rearrange the molecules. If the manufacturer changes the formulation tomorrow, you don't care — the interface (swallow) stays the same.

3.1 Access Modifiers

MUST KNOW **GFG CLASSIC**

Modifier	Same class	Same package	Subclass	Anywhere
private	Yes	No	No	No
default (Java, no modifier)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

C++ note: protected in C++ allows subclass access regardless of package (no packages). **Python convention:** no enforced access — `_name` is "protected by convention", `__name` triggers name mangling. Python trusts the developer.

3.2 Getters & Setters

HIGH FREQ

The standard pattern for safe encapsulation: make fields private, expose them via public methods called getters and setters.

```
class BankAccount {  
    private double balance;           // hidden  
  
    public double getBalance() {      // controlled read  
        return balance;  
    }  
  
    public void deposit(double amt) { // controlled write  
        if (amt <= 0) throw new IllegalArgumentException();  
        balance += amt;  
    }  
}
```

★ **Interview Gold: Why bother with getters/setters if they just expose the field?** Three reasons:
(1) **Validation** — reject invalid input (negative deposit).
(2) **Change freedom** — you can change balance from double to BigDecimal without breaking callers.

(3) **Side effects** — log access, fire events, lazy-load.
Direct field access locks you in forever.

3.3 Immutability

AMAZON FAV TRICKY

An **immutable** object can't be modified after creation. Its fields are set in the constructor and never changed. Examples: String in Java, all Python tuples, all primitive wrappers.

To make a class immutable:

1. Mark all fields final (Java) / const (C++).
2. Don't provide setters.
3. Set all fields in the constructor.
4. Make the class final so it can't be subclassed and weakened.
5. If a field is mutable itself (e.g., a list), return a defensive copy from the getter.

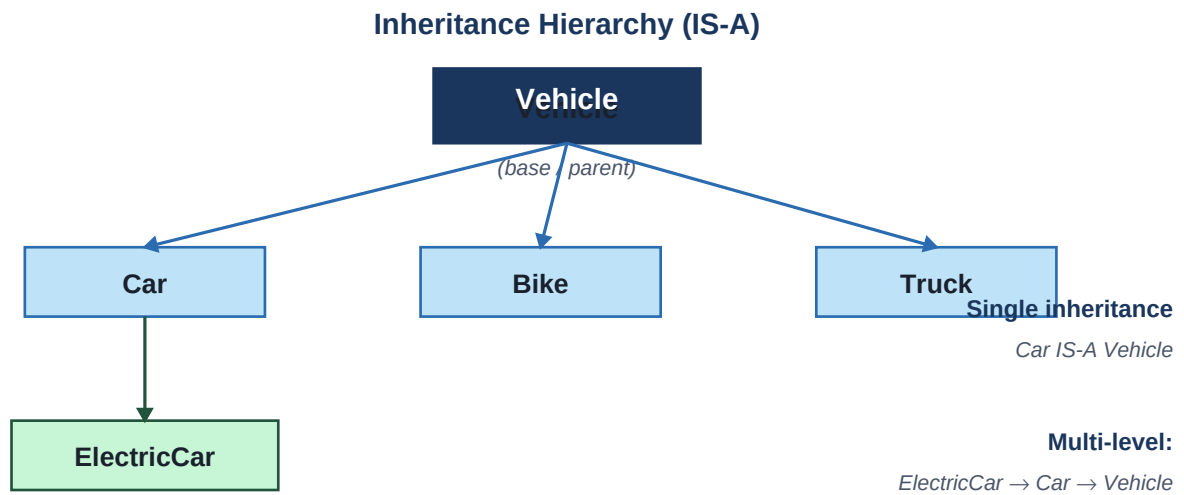
★ **Interview Gold: Why immutability matters:** thread-safe by default (no one can change it mid-read), safe as map keys, easy to reason about, no defensive copying needed when passing around. Modern Java records, Kotlin data classes, and functional languages lean heavily on immutability.

4. Inheritance

MUST KNOW HIGH FREQ AMAZON FAV SERVICE-BASED FAV

Inheritance lets a class derive state and behavior from another class. The derived class (subclass / child) gets everything the base class (superclass / parent) has, plus whatever it adds or overrides. The relationship is **IS-A**.

■ **Analogy:** A Tesla IS-A Car, which IS-A Vehicle. Each level adds capabilities — Vehicle gives you wheels and speed, Car adds doors and seats, Tesla adds Autopilot and OTA updates. You inherit your parents' DNA and add your own life on top — same idea.



4.1 Types of inheritance

HIGH FREQ GFG CLASSIC

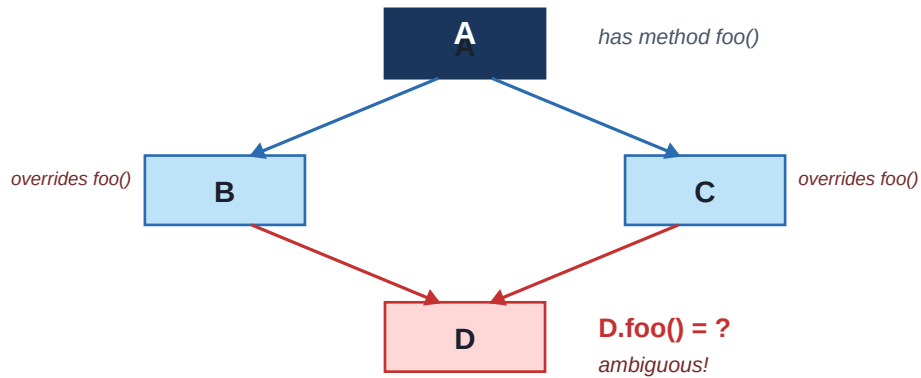
Type	Shape	Supported in
Single	$A \rightarrow B$	All OOP languages
Multi-level	$A \rightarrow B \rightarrow C$	All OOP languages
Hierarchical	$A \rightarrow B, A \rightarrow C, A \rightarrow D$	All OOP languages
Multiple	$A \rightarrow C, B \rightarrow C$ (C extends both A and B)	C++, Python. NOT Java/C# (use interfaces)
Hybrid	Mix of the above (e.g., diamond)	C++ with virtual inheritance

4.2 The Diamond Problem

HIGH FREQ TRICKY GFG CLASSIC

The classic problem with multiple inheritance. If B and C both inherit from A and both override `foo()`, and D inherits from B and C — which `foo()` does D get? This ambiguity is why Java and C# banned multiple class inheritance.

The Diamond Problem (Multiple Inheritance)



C++ allows it (resolved via virtual inheritance). Java/C# disallow — use interfaces.

★ Interview Gold: How languages handle it:

- **C++:** allows it; you fix ambiguity manually or use virtual inheritance so A is shared.
 - **Java / C#:** forbidden for classes. Use *multiple interface implementation* instead — interfaces have no state, so no ambiguity.
 - **Python:** allows it; uses **MRO (Method Resolution Order)** via the C3 linearization algorithm.
- If you're asked "why doesn't Java support multiple inheritance?", the answer is "to avoid the diamond problem".

4.3 Syntax across languages

Language	Syntax
Java	<code>class Car extends Vehicle implements Drivable { }</code>
C++	<code>class Car : public Vehicle { };</code>
Python	<code>class Car(Vehicle): ...</code>
C#	<code>class Car : Vehicle, IDrivable { }</code>

4.4 super and base-class access

Inside a subclass, you often need to call the parent's version of a method or constructor — `super` is the keyword. In C++ it's `BaseName::method()`.

```
class Animal {
    void greet() { System.out.println("Hello"); }
}
class Dog extends Animal {
    void greet() {
        super.greet();           // call parent's greet first
        System.out.println("Woof");
    }
}
```

4.5 Composition over Inheritance

AMAZON FAV RARE BUT IMPRESSIVE

Modern design wisdom: **prefer composition (HAS-A) over inheritance (IS-A)** where possible. Inheritance creates tight coupling — changes to the base class ripple through every subclass. Composition is more flexible: hold a reference to another object and delegate.

Composition (HAS-A) vs Inheritance (IS-A)



Modern advice: "favor composition over inheritance" — more flexible, less coupling.

```
// Inheritance – Car IS-A Vehicle
class Car extends Vehicle { }

// Composition – Car HAS-A Engine
class Car {
    private Engine engine;           // holds another object
    public void start() {
        engine.ignite();             // delegate
    }
}
```

★ **Interview Gold:** Composition lets you **swap implementations at runtime** — change the Engine without changing the Car class. It's also the foundation of **Dependency Injection** and most modern design patterns (Strategy, Decorator). Inheritance is a one-time, compile-time commitment.

5. Polymorphism

MUST KNOW **HIGH FREQ** **AMAZON FAV** **SERVICE-BASED FAV**

Polymorphism = "**many forms**". One name (method, operator) behaves differently based on the context. Two flavors: **compile-time** (method overloading, operator overloading) and **runtime** (method overriding via inheritance).

■ **Analogy:** The verb "run" is polymorphic. I run a marathon, I run a company, I run a program, my nose is running. Same word, very different actions — interpreted from context. Polymorphism in code is exactly that: same call, different behavior depending on the actual object type.

5.1 Compile-time Polymorphism (Overloading)

HIGH FREQ **GFG CLASSIC**

Method overloading: multiple methods with the *same name* but different parameter lists in the same class. The compiler picks which one to call based on the arguments you pass.

```
class Calculator {  
    int add(int a, int b)           { return a + b; }  
    double add(double a, double b) { return a + b; }  
    int add(int a, int b, int c)    { return a + b + c; }  
}  
// Compiler resolves which add() to call at compile time.
```

★ **Interview Gold: Overloading rules:**

- Methods must differ in **number, type, or order of parameters**.
- Return type alone is **NOT** enough — `int f()` and `double f()` won't compile.
- Access modifier and exceptions can differ freely.

Constructors can be overloaded too — same rules apply.

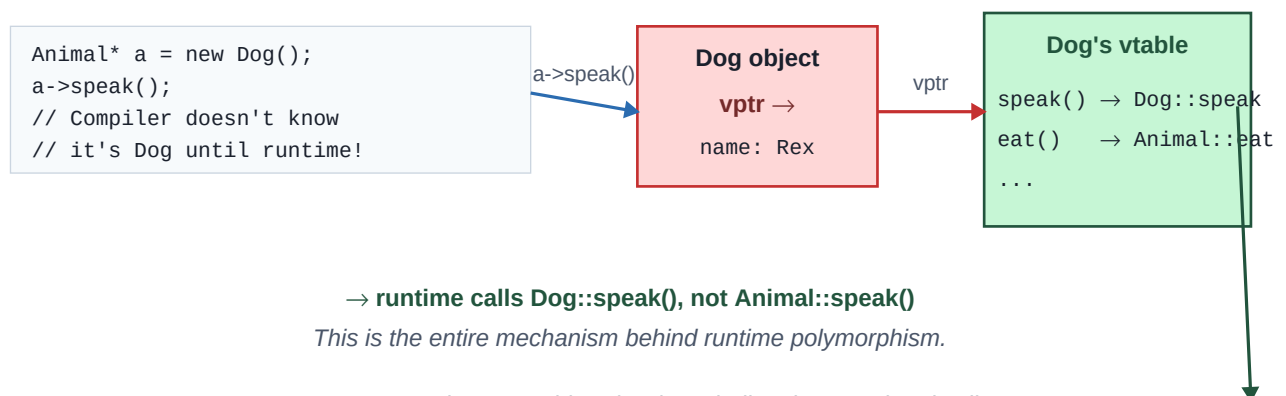
5.2 Runtime Polymorphism (Overriding)

MUST KNOW **AMAZON FAV**

Method overriding: a subclass redefines a method that exists in its parent. When you call the method on a base-class reference that actually points to a derived object, the **derived class's version runs** — resolved at runtime.

```
class Animal {  
    void speak() { System.out.println("Some sound"); }  
}  
class Dog extends Animal {  
    @Override  
    void speak() { System.out.println("Woof"); }  
}  
  
Animal a = new Dog();  
a.speak(); // prints "Woof" – Dog's version, decided at runtime
```

Runtime Polymorphism — Dynamic Dispatch via vtable



5.3 How runtime dispatch works — vtables

RARE BUT IMPRESSIVE

Every class with virtual/overridable methods gets a **vtable** — an array of function pointers, one per virtual method. Every object stores a hidden **vp**tr pointing to its class's vtable. When you call a virtual method, the runtime:

- (1) follows the object's vp
tr to the vtable,- (2) looks up the method's slot,
- (3) calls the function at that address.

★ **Interview Gold: Why this matters in interviews:** When asked "what's the cost of polymorphism?", the answer is *one extra pointer per object (vp*tr*) and one indirection per virtual call*. It's why C++ makes you explicitly mark methods virtual — only those go in the vtable. Java/C# make every method virtual by default (and use final to opt out).

5.4 Overloading vs Overriding — the comparison table

MUST KNOW HIGH FREQ GFG CLASSIC TRICKY

Aspect	Overloading	Overriding
When resolved	Compile time	Runtime
Where	Same class (usually)	Across parent → child
Signature	Must differ	Must match exactly
Return type	Can differ (but doesn't help resolution)	Same or covariant
Access modifier	Free to differ	Can't be more restrictive than parent
Inheritance needed?	No	Yes — overriding requires base method
Type of polymorphism	Compile-time / static	Runtime / dynamic

5.5 The @Override annotation

In Java, marking a method @Override tells the compiler "this is supposed to override a parent method". If you typo the method name or signature, the compiler errors out instead of silently creating a new, unrelated method. Always use it — it catches real bugs.

★ **Interview Gold: Quick traps:** Static methods are *hidden*, not overridden. Private methods can't be overridden. final methods can't be overridden. Constructors can be overloaded but never overridden. Java has no user-defined operator overloading (except + for String); C++ and Python do.

6. Abstraction

MUST KNOW HIGH FREQ TRICKY

Abstraction = show what something does, hide how it does it. You expose a minimal, useful interface; you keep the messy details inside. Done via **abstract classes** and **interfaces**.

■ **Analogy:** A TV remote is an abstraction. The button says "volume up" — you press it, volume goes up. You don't know about IR pulses, signal protocols, the DSP inside the TV. The remote is a tiny, useful

API hiding electrical engineering behind it. Good abstractions make complex systems usable.

★ **Interview Gold: Encapsulation vs Abstraction** — the most-asked tricky distinction in OOP:

- **Encapsulation** is a *mechanism* — bundling data with methods and using access modifiers.
- **Abstraction** is a *design goal* — exposing only what's necessary.

Encapsulation is HOW you achieve abstraction. They're related but not the same.

6.1 Abstract Classes

HIGH FREQ

An **abstract class** is a class you can't instantiate directly — it exists to be extended. It can have fully implemented methods AND abstract methods (declared but unimplemented). Subclasses must implement all abstract methods (or be abstract themselves).

```
abstract class Shape {
    String color;
    abstract double area();           // must be implemented by subclass
    void describe() {                 // fully implemented – shared
        System.out.println("A " + color + " shape with area " + area());
    }
}
class Circle extends Shape {
    double r;
    double area() { return Math.PI * r * r; }
}
```

6.2 Interfaces

HIGH FREQ

AMAZON FAV

An **interface** is a contract — a list of methods a class promises to implement. Traditionally interfaces have no state and no implementation, just signatures. Java 8+ added default methods (with bodies) and static methods.

```
interface Drawable {
    void draw();                       // abstract by default
    default void erase() {              // Java 8+: default impl
        System.out.println("Erasing...");
    }
}
class Circle implements Drawable {
    public void draw() { /* render circle */ }
}
```

6.3 Abstract Class vs Interface

MUST KNOW

TRICKY

GFG CLASSIC

Abstract Class vs Interface

Abstract Class

- Can have state (fields)
- Can have constructors
- Can have full method bodies
- Can have abstract methods too
- Single inheritance only
- Use when: "IS-A" + shared code
- Example: Shape (base for Circle, Square — shares color/area logic)

Interface

- No state (only constants*)
- No constructors
- Method signatures only (mostly)*
- All methods implicitly abstract*
- Multiple implementation allowed
- Use when: "CAN-DO" capability
- Example: Comparable, Iterable, Serializable, Cloneable

* Java 8+ allows default and static methods in interfaces; constants are always public static final.

Aspect	Abstract Class	Interface
State (fields)	Yes, instance fields allowed	No state (only constants in Java)
Constructors	Yes	No
Method bodies	Yes — both abstract and concrete	Mostly signatures (default methods in Java 8+)
Multiple inheritance	No (single base class)	Yes — implement many interfaces
Access modifiers	All allowed	All methods public in Java
When to use	Share code AND require subclass implementation	Define a capability/contract
Says	"IS-A"	"CAN-DO"

★ Interview Gold: The mental rule:

- Use an **abstract class** when subclasses share *code* as well as a type — e.g., several Shapes sharing color/border but each computing area differently.
- Use an **interface** when classes share only a *capability* — e.g., totally different classes (File, NetworkConnection, DBConnection) all being Closeable.

Java standard library: Comparable, Iterable, Runnable — all capabilities → interfaces. AbstractList, AbstractMap — shared partial implementations → abstract classes.

6.4 Programming to an Interface

AMAZON FAV RARE BUT IMPRESSIVE

"Program to an interface, not an implementation" — the most quoted line in the Gang of Four design patterns book. Means: declare variables, parameters, and return types using interfaces (List) rather than concrete classes (ArrayList). Lets you swap implementations later.

```
// Bad: locked into ArrayList
ArrayList<String> users = new ArrayList<>();

// Good: any List works
List<String> users = new ArrayList<>();    // could become LinkedList tomorrow
```

7. SOLID Principles

HIGH FREQ AMAZON FAV RARE BUT IMPRESSIVE

SOLID is five design principles for writing maintainable OOP code, coined by Robert C. Martin ("Uncle Bob"). Each letter is one principle. Interviewers love these — knowing them, even at the surface level, signals you read past tutorials.

★ **Interview Gold:** Memory aid: **S-O-L-I-D** = Single, Open/closed, Liskov, Interface segregation, Dependency inversion. Be able to give a one-line definition and a one-line example for each. That's enough for fresher interviews.

7.1 S — Single Responsibility Principle (SRP)

MUST KNOW

A class should have one, and only one, reason to change. Each class should do one thing — if you find yourself describing it with "and", it probably violates SRP.

```
// Bad: this class does TWO things
class User {
    void save() { /* writes to DB */ }
    void sendEmail() { /* SMTP logic */ }
}

// Good: split responsibilities
class User { /* data only */ }
class UserRepository { void save(User u) { } }
class EmailService { void send(User u, String msg) { } }
```

■ **Analogy:** SRP is like a Swiss Army knife vs a chef's kitchen. A Swiss Army knife technically opens bottles, cuts string, and clips nails — but a kitchen has a separate bottle opener, knife, and scissors, each great at one job. Changing one doesn't affect the others.

7.2 O — Open/Closed Principle (OCP)

HIGH FREQ

Classes should be open for extension, closed for modification. You should be able to add new behavior without changing existing code — typically by inheriting or composing, not by editing tested code.

```
// Bad: every new shape requires modifying this method
double area(Shape s) {
    if (s instanceof Circle)    return ...;
    if (s instanceof Square)    return ...;
    if (s instanceof Triangle)  return ...;
}

// Good: each Shape implements its own area()
interface Shape { double area(); }
class Circle implements Shape { public double area() { ... } }
class Square implements Shape { public double area() { ... } }
// Adding Pentagon needs no changes to existing code
```


7.3 L — Liskov Substitution Principle (LSP)

TRICKY GFG CLASSIC

Subtypes must be substitutable for their base types without breaking the program. Anywhere code expects an Animal, you should be able to pass a Dog and everything should still work correctly.

Classic violation: **Square extends Rectangle**. Sounds fine geometrically — but:

```
class Rectangle {
    int w, h;
    void setWidth(int w) { this.w = w; }
    void setHeight(int h) { this.h = h; }
}
class Square extends Rectangle {
    void setWidth(int w) { this.w = w; this.h = w; } // forces both
    void setHeight(int h) { this.w = h; this.h = h; }
}

// Code that assumes Rectangle's contract breaks:
Rectangle r = new Square();
r.setWidth(5);
r.setHeight(10);
assert r.getArea() == 50; // FAILS - area is 100, not 50
```

★ **Interview Gold: Why this is the trickiest principle:** it's about *behavioral compatibility*, not just type compatibility. The compiler is happy that Square IS-A Rectangle, but the *behavior* changed. LSP says: if you can break callers by substituting, your inheritance is wrong. **Solution:** don't model Square as a Rectangle subclass. Make them separate, or use composition.

7.4 I — Interface Segregation Principle (ISP)

Don't force classes to depend on methods they don't use. Many small, focused interfaces beat one large "fat" interface.

```
// Bad: every Worker must implement eat(), even robots
interface Worker {
    void work();
    void eat();
}

// Good: split
interface Workable { void work(); }
interface Feedable { void eat(); }
class Human implements Workable, Feedable { }
class Robot implements Workable { }
```

7.5 D — Dependency Inversion Principle (DIP)

AMAZON FAV RARE BUT IMPRESSIVE

Depend on abstractions, not on concrete implementations. High-level modules shouldn't know about the low-level classes they use — both should depend on an interface.

```
// Bad: NotificationService is hard-coded to email
class NotificationService {
    EmailSender sender = new EmailSender(); // tight coupling
    void notify(...) { sender.send(...); }
}
```

```
// Good: depend on abstraction; inject at construction
interface MessageSender { void send(String msg); }
class NotificationService {
    private MessageSender sender;
    NotificationService(MessageSender s) { this.sender = s; }
    void notify(...) { sender.send(...); }
}
// Now you can swap EmailSender, SMSSender, SlackSender – no code changes
```

★ **Interview Gold: DIP is the foundation of Dependency Injection (DI)**, used by Spring, Angular, ASP.NET, and pretty much every modern backend framework. The idea: instead of an object creating its own dependencies, hand them in from outside. This is also what makes code testable — pass in a mock instead of a real database.

7.6 Quick recap

Letter	Name	One-line summary
S	Single Responsibility	One class, one job.
O	Open/Closed	Open to extend, closed to modify.
L	Liskov Substitution	Subclass behaves correctly where the base is expected.
I	Interface Segregation	Small focused interfaces beat fat ones.
D	Dependency Inversion	Depend on abstractions, not concretions.

8. Common Design Patterns

HIGH FREQ AMAZON FAV

Design patterns are reusable solutions to common OOP design problems. There are 23 classic Gang-of-Four patterns; freshers are typically asked about ~5–6. Know these well; skip the obscure ones. Each below has the *problem*, *solution*, and *minimal example*.

8.1 Pattern categories

Category	Purpose	Examples
Creational	How objects get created	Singleton, Factory, Builder
Structural	How objects are composed	Adapter, Decorator, Facade, Proxy
Behavioral	How objects communicate	Observer, Strategy, Iterator, Command

8.2 Singleton

MUST KNOW HIGH FREQ GFG CLASSIC TRICKY

Problem: Some classes should have *exactly one instance* — a config registry, a logger, a connection pool.

Solution: Make the constructor private and provide a static accessor that always returns the same instance.

```
// Classic (not thread-safe)
class Singleton {
    private static Singleton instance;
    private Singleton() { } // private!
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}

// Thread-safe (lazy + double-checked locking)
class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

★ **Interview Gold:** Singleton is **controversial**. Modern view: it's a glorified global variable, hurts testability, and is hard to do right (thread safety, lazy init, serialization, reflection bypass). In modern code, prefer **dependency injection** — pass the single instance in from outside. Interviewers ask Singleton to test if you can write thread-safe code, not because they love the pattern.

8.3 Factory Method

HIGH FREQ

Problem: Creating objects directly couples your code to concrete classes.

Solution: Move object creation into a method (a factory). Callers ask the factory for what they want; the factory decides which concrete class to instantiate.

```
interface Animal { void speak(); }
class Dog implements Animal { public void speak() { /* woof */ } }
class Cat implements Animal { public void speak() { /* meow */ } }

class AnimalFactory {
    public static Animal create(String type) {
        if (type.equals("dog")) return new Dog();
        if (type.equals("cat")) return new Cat();
        throw new IllegalArgumentException();
    }
}

Animal a = AnimalFactory.create("dog"); // caller doesn't know about Dog
```

8.4 Observer

HIGH FREQ AMAZON FAV

Problem: When one object changes state, several others need to react — without tight coupling.

Solution: The subject keeps a list of observers and notifies all of them when its state changes.

```
interface Observer { void update(String event); }

class Subject {
    List<Observer> observers = new ArrayList<>();
    void subscribe(Observer o) { observers.add(o); }
    void notifyAll(String event) {
        for (Observer o : observers) o.update(event);
    }
}
```

■ **Analogy:** Observer is exactly how a YouTube channel works. The channel (subject) doesn't know who its subscribers are individually — it just maintains a list and notifies everyone on a new upload. Subscribers can join or leave freely. The mechanism is the same in code: pub/sub event systems, JavaScript's `addEventListener`, RxJS observables — all observer-pattern descendants.

8.5 Strategy

Problem: A class needs to switch between several algorithms at runtime, and you don't want if/else chains.

Solution: Define a family of algorithms behind a common interface; the class holds a reference to one and delegates.

```
interface PaymentStrategy { void pay(double amt); }
class CreditCard implements PaymentStrategy { public void pay(double amt) { ... } }
class UPI implements PaymentStrategy { public void pay(double amt) { ... } }

class Cart {
    private PaymentStrategy strategy;
    void checkout(double total) { strategy.pay(total); }
    void setStrategy(PaymentStrategy s) { strategy = s; }
}
```

★ **Interview Gold: Strategy is composition over inheritance in action.** Instead of CreditCardCart, UPICart, CashCart subclasses (inheritance), you have one Cart holding a swappable strategy (composition). Far more flexible.

8.6 Decorator

RARE BUT IMPRESSIVE

Problem: Add behavior to an object dynamically without modifying its class or creating a subclass for every combination.

Solution: Wrap the object in another object (the decorator) that implements the same interface and adds behavior.

```
interface Coffee { double cost(); }
class SimpleCoffee implements Coffee { public double cost() { return 5; } }

abstract class CoffeeDecorator implements Coffee {
    protected Coffee inner;
    CoffeeDecorator(Coffee c) { inner = c; }
}

class WithMilk extends CoffeeDecorator { WithMilk(Coffee c) { super(c); } public double cost() { return inner.cost() + 1; } }
```

```
class WithSugar extends CoffeeDecorator { WithSugar(Coffee c) { super(c); } public double cost() { return inner.cost() + 0.5; } }
```

```
Coffee c = new WithSugar(new WithMilk(new SimpleCoffee())); // cost = 6.5
```

★ **Interview Gold:** Java's I/O streams are decorator pattern in the wild: `new BufferedReader(new InputStreamReader(System.in))` is a chain of decorators each adding capability. You'll see this all over real codebases.

8.7 When to know which pattern

- **Singleton:** when exactly one instance is needed AND state must be globally accessible.
- **Factory:** when creation logic is complex or the concrete type depends on input.
- **Observer:** any pub/sub or event-driven situation.
- **Strategy:** when an algorithm should be selectable at runtime.
- **Decorator:** when you'd otherwise need a combinatorial explosion of subclasses.

★ **Interview Gold: Don't over-apply patterns.** Junior developers love discovering patterns and forcing them everywhere — it makes simple code complicated. The right answer to "which pattern should I use?" is often "none — just write a normal class". Patterns are vocabulary for when complexity demands it, not stickers to slap on everything.

9. OOP Across Languages

HIGH FREQ TRICKY

OOP looks slightly different in C++, Java, and Python. Interviewers love asking "why does X work in Python but not in Java?". Know the key differences and you'll handle these cleanly.

9.1 Quick comparison

Feature	C++	Java	Python
Multiple inheritance	Yes (virtual)	No (classes); Yes (interfaces)	Yes (with MRO)
Access control	Compiler-enforced	Compiler-enforced	Convention (<code>_</code> , <code>__</code>)
Memory management	Manual (new/delete) or RAII	Garbage collected	Reference counted + GC
Virtual methods	Opt-in (virtual keyword)	Default; opt-out via final	All methods are virtual
Interfaces	Pure abstract classes	interface keyword	ABCs or duck typing
Operator overloading	Yes, extensive	No (limited)	Yes (dunder methods)
Default values	Allowed in functions	Not in methods (overload)	Yes
Constructor chaining	<code>: Base()</code>	<code>super()</code>	<code>super().__init__()</code>

9.2 C++ peculiarities

- **Multiple inheritance** is allowed and used (e.g., the I/O stream hierarchy). Diamond problem solved via virtual inheritance.
- **Friends**: friend functions and classes can access private members. No equivalent in Java/Python.
- **Destructors** are deterministic — run when an object goes out of scope (RAII). This is huge for resource management.
- **Virtual is opt-in**: virtual void foo(). Non-virtual methods are statically dispatched and cheaper.
- **Templates** provide compile-time polymorphism that's orthogonal to inheritance.

9.3 Java peculiarities

- **Everything in a class** (no free functions). Static methods live inside classes too.
- **All methods virtual by default**; use final to forbid overriding.
- **No multiple class inheritance** — only multiple interface implementation.
- **Garbage collected** — you never delete. Closing resources is done via try-with-resources.
- **equals() and hashCode()**: must be overridden together. If two objects are equal, they must have the same hashCode. (**High frequency interview question.**)
- **Records (Java 14+)**: compact syntax for immutable data classes — auto-generated equals, hashCode, toString.

★ **Interview Gold: The equals() / hashCode() contract** shows up in almost every Java interview. Rule: *if a.equals(b) is true, then a.hashCode() == b.hashCode() must be true*. The reverse is not required (two objects can have the same hashCode without being equal — that's just a collision). Break this contract and HashMap/HashSet will silently misbehave.

9.4 Python peculiarities

- **No real access control**. Everything is public; `__name` is a convention "don't touch"; `__name` triggers name mangling (becomes `_ClassName__name`) — still reachable.
- **Duck typing**: "if it walks like a duck and quacks like a duck..." — type doesn't need to match, only the methods/attributes you actually use.
- **Every method is virtual**. All methods can be overridden; dispatch is dynamic.
- **Multiple inheritance with MRO** (C3 linearization). `super()` walks the MRO order, not just the parent.
- **`__init__` is a method, not a constructor**. The actual constructor is `__new__`; `__init__` just initializes the already-created instance.
- **Dunder methods** drive operator overloading and built-in protocols: `__add__`, `__len__`, `__iter__`, `__eq__`, `__hash__`, `__enter__` / `__exit__`.

9.5 The same code in three languages

A simple class with a method, in all three:

```
// C++
class Animal {
public:
    std::string name;
```

```
Animal(std::string n) : name(n) {}  
virtual void speak() { std::cout << "..."; }  
};
```

```
// Java  
class Animal {  
    String name;  
    Animal(String name) { this.name = name; }  
    void speak() { System.out.println("..."); }  
}
```

```
# Python  
class Animal:  
    def __init__(self, name):  
        self.name = name  
    def speak(self):  
        print("...")
```

10. Interview Rapid-Fire

Last-day revision pack. Read it once the night before. Each section is built for fast scanning, not deep reading.

10.1 Top 30 fresher OOPS questions

Concepts (10)

1. **What is OOP?** A paradigm organizing code around objects that bundle state and behavior.
2. **Four pillars?** Encapsulation, Inheritance, Polymorphism, Abstraction.
3. **Class vs Object?** Class = blueprint, Object = instance in memory.
4. **What is encapsulation?** Bundling data + methods, hiding internal state behind a public interface.
5. **What is abstraction?** Showing only essential features; hiding implementation complexity.
6. **Encapsulation vs Abstraction?** Encapsulation is the mechanism; abstraction is the goal.
7. **What is inheritance?** A class deriving fields/methods from another; IS-A relationship.
8. **What is polymorphism?** One name, many forms; resolved at compile-time (overloading) or runtime (overriding).
9. **Types of inheritance?** Single, multi-level, hierarchical, multiple, hybrid.
10. **Composition vs inheritance?** HAS-A vs IS-A. Prefer composition for flexibility.

Classes & Objects (5)

1. **What's a constructor?** Special method run when an object is created; initializes state.
2. **Can a constructor return a value?** No — it implicitly returns the new object.
3. **What is this?** Reference to the current object inside an instance method.
4. **What does static mean?** Belongs to the class, not instances. Shared across objects.
5. **Can a static method use this?** No — no specific object to refer to.

Inheritance & Polymorphism (8)

- 1. Overloading vs Overriding?** Overloading = same name, different params, compile-time. Overriding = subclass redefines parent method, runtime.
- 2. Can you override a static method?** No — it's hidden, not overridden.
- 3. Can you override a private method?** No — not visible to subclasses.
- 4. What's super?** Reference to immediate parent class; used to call parent constructor or method.
- 5. Diamond problem?** Ambiguity in multiple inheritance when two parents have the same method.
- 6. How does Java avoid the diamond problem?** By not allowing multiple class inheritance; uses interfaces.
- 7. What's a virtual function?** A method that's resolved at runtime via vtable lookup.
- 8. What's the vtable?** Array of function pointers per class; one per virtual method.

Abstract / Interface (4)

- 1. Abstract class vs Interface?** Abstract class can have state and method bodies; interface is mostly a pure contract. Use abstract for shared code, interface for capability.
- 2. Can an abstract class have a constructor?** Yes — called by subclass via super().
- 3. Can an interface extend a class?** No. An interface can only extend other interfaces.
- 4. What's a marker interface?** An empty interface used as a tag (e.g., Serializable).

Patterns & Principles (3)

- 1. SOLID full form?** Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
- 2. Singleton pattern in one line?** Restrict a class to one instance via a private constructor and static accessor.
- 3. Observer pattern in one line?** Subject notifies a list of subscribed observers when its state changes.

10.2 The trickiest distinctions

TRICKY MUST KNOW

This vs That	The crisp difference
Class vs Object	Blueprint vs Instance. Class lives in source; object lives at runtime.
Encapsulation vs Abstraction	Mechanism (bundling) vs Goal (hiding complexity).
Abstract class vs Interface	Has state & partial code vs pure contract. IS-A vs CAN-DO.
Overloading vs Overriding	Compile-time, same class vs Runtime, parent-child.
Composition vs Inheritance	HAS-A vs IS-A. Composition is more flexible.
Static vs Instance	Class-level (one copy) vs Per-object.
Method hiding vs Overriding	Static method hiding vs Instance method overriding.

10.3 The 3-minute pre-interview revision

If you have 3 minutes before the interview, reread just this:

1. **Four pillars:** Encapsulation, Inheritance, Polymorphism, Abstraction.
2. **Class** is blueprint; **object** is instance.
3. **Overloading:** same name, different params, compile-time.
4. **Overriding:** subclass redefines parent method, runtime via vtable.
5. **Abstract class:** partial implementation; **interface:** pure contract. Java: only multiple interface inheritance.
6. **Diamond problem:** multiple inheritance ambiguity; Java avoids via interfaces, C++ via virtual inheritance, Python via MRO.
7. **SOLID:** Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion.
8. **Singleton:** private constructor + static accessor; thread-safe needs double-checked locking.
9. **Favor composition over inheritance.**
10. **equals() and hashCode()** must be overridden together (Java).

★ **Interview Gold: Last advice:** when asked to *design* something in OOP ("design a parking lot", "design a library system"), always think: *what are the objects, what does each one know, what does each one do?* Identify classes from nouns, methods from verbs. Use composition where things **HAVE** another thing, inheritance where things **ARE** another thing. Don't over-engineer — interviewers want clarity, not pattern showcases.

— End of OOPS Placement Prep —